

Proyecto I: Reconocimiento de comandos de voz y despliegue móvil con ONNX Runtime Mobile

Fabrizio Mena Mejía

Instituto Tecnológico de Costa Rica

Email: fabriciomena11@estudiantec.cr

Anthony Fuentes Calvo

Instituto Tecnológico de Costa Rica

Email: anthonyfuentescalvo@estudiantec.cr

Fernando Sánchez Hidalgo

Instituto Tecnológico de Costa Rica

Email: Fer.sanchez@estudiantec.cr

Abstract—Este informe corresponde al Proyecto I del curso de Inteligencia Artificial: se construye, de extremo a extremo, un reconocedor de comandos de voz de vocabulario reducido orientado a un escenario de asistencia o control (p.ej. hogar inteligente), pasando del dato crudo al despliegue en dispositivo. Siguiendo el enunciado, se adopta el conjunto público *Google Speech Commands* v0.02 con diez clases, partición oficial de entrenamiento, validación y prueba, y una representación tipo imagen mediante mel-espectrograma normalizado y reescalado a 128×128 para alimentar redes convolucionales en PyTorch. Se comparan dos familias de modelos—LeNet-5 adaptado (con Tanh) y MobileNetV2 en una variante ligera—bajo datos crudos frente a datos aumentados (desplazamiento temporal, ruido y SpecAugment) y tres configuraciones de hiperparámetros por combinación, lo que produce doce entrenamientos comparables. Las métricas en prueba muestran superioridad sostenida de MobileNetV2 (hasta 0.9633 de exactitud en la mejor corrida). El seguimiento experimental se centralizó en Weights & Biases. Por último, se documenta la línea de trabajo hacia la exportación ONNX, la comprobación de paridad con respecto a PyTorch y la integración prevista con ONNX Runtime Mobile en una aplicación Android (Jetpack Compose) que mapea la predicción a acciones de interfaz.

Index Terms—Comandos de voz, mel-espectrograma, redes neuronales convolucionales, aumentación de datos, Weights & Biases, ONNX, ONNX Runtime Mobile.

I. MARCO TEÓRICO

La clasificación de audio corto puede formularse como reconocimiento de patrones en una imagen tiempo-frecuencia: a partir de una forma de onda muestreada se calcula una transformada de Fourier de tiempo corto (STFT) y un banco de filtros mel que aproxima la percepción humana de tono; el logaritmo de la energía mel amortigua la dinámica y facilita el entrenamiento de redes profundas [6]. Las redes neuronales convolucionales (CNN) en dos dimensiones extraen jerarquías locales de texturas y formas en ese mapa, de forma análoga a la visión por computador aplicada a representaciones espectrales.

Para mejorar la generalización bajo ruido y variaciones de canal, es habitual aplicar aumentación en el dominio del tiempo (p.ej. desplazamiento) y en el espectro; SpecAugment enmascara franjas de tiempo y frecuencia en el mel-espectrograma durante el entrenamiento, obligando al modelo a no depender de un único bin o frame [1].

En dispositivos móviles, conviene separar entrenamiento (PyTorch) e inferencia ligera: exportar el grafo a ONNX en modo evaluación, con pesos congelados, y ejecutarlo con

ONNX Runtime Mobile reduce dependencias y habilita optimizaciones de tiempo de ejecución [2]. La calidad del despliegue exige que el preprocesamiento en el terminal replique fielmente el usado al entrenar; la discrepancia se controla con pruebas de paridad logit-a-logit entre PyTorch y el runtime ONNX [2].

II. DATASET Y PREPROCESAMIENTO

A. *Speech Commands* v0.02 y subconjunto de diez clases

Se utilizó el conjunto *Speech Commands* v0.02 distribuido por TensorFlow Datasets [5], compuesto por grabaciones en formato WAV de un segundo o menos, organizadas en carpetas por etiqueta léxica. Siguiendo el alcance del proyecto, se trabajó con diez comandos: yes, no, up, down, left, right, on, off, stop y go.

B. División entrenamiento, validación y prueba

La partición oficial del corpus se respeta mediante los archivos `validation_list.txt` y `testing_list.txt` incluidos en la raíz del dataset: las rutas listadas en ellos definen los conjuntos de validación y de prueba, respectivamente. El entrenamiento comprende el resto de archivos `.wav` de las diez clases que no aparecen en dichas listas, lo que garantiza que no haya solapamiento de hablantes ni clips entre particiones y que los resultados sean comparables con la literatura.

C. Representación tipo imagen y flujo reproducible

Cada clip se re-muestra a mono PCM de 16 kHz y se recorta o rellena a una ventana fija de 1 s (16 000 muestras) antes del análisis espectral. Para asegurar consistencia entre entrenamiento y despliegue móvil, se fija un **único contrato de preprocesamiento** (usado para exportación ONNX y para inferencia en Android): STFT con ventana Hann de 25 ms (`win_length=400`), `n_fft=512` y `hop_length=160` (10 ms); banco mel HTK de `n_mels=64` entre 20–8000 Hz; y mel-espectrograma en escala logarítmica $\log(\text{mel} + \epsilon)$. El mapa (64×101) resultante se re-escala bilinealmente a 128×128 y se normaliza para formar el tensor de entrada en NCHW (1,1,128,128), compatible con las entradas descritas en las Tablas I y II. Este contrato se implementa de manera equivalente en el cuaderno de entrenamiento (`proy1.ipynb`) y en la aplicación móvil, evitando discrepancias silenciosas entre PyTorch y ONNX Runtime Mobile.

La reproducibilidad se apoya en la semilla global 42 (véase el control de aleatoriedad en la siguiente sección), en el uso determinista de las listas oficiales de validación y prueba y en la trazabilidad de hiperparámetros y curvas en *Weights & Biases*.

III. DISEÑO DE ARQUITECTURA DE IA

A. Control de Aleatoriedad y Reproducibilidad

Para garantizar la reproducibilidad de los experimentos, se implementó un estricto control de aleatoriedad. Se definió una semilla global (*seed* = 42) utilizada para inicializar los generadores de números pseudoaleatorios de las librerías *random*, *numpy* y *torch*. Adicionalmente, se configuró el comportamiento determinista de *cuDNN* desactivando el *benchmark* dinámico donde fuera necesario.

B. Selección de Técnicas de Aumentación de Datos

Para mejorar la capacidad de generalización de los modelos y simular condiciones realistas de entorno (ruido de fondo, variabilidad de hablantes), se aplicaron técnicas de aumento de datos (*data augmentation*) en el preprocesamiento de audio. Las técnicas se inspiraron en métodos avanzados de aumento espectral y en el dominio del tiempo:

- **Time Shift:** Desplazamiento temporal aleatorio para que el sistema aprenda predicciones invariantes al milisegundo exacto de reproducción, alineándose directamente con aplicaciones reales bajo ruidos dispares.
- **Ruido Blanco (White Noise):** Inyectado progresivamente para simular grabaciones en ambientes no controlados y servir de fuerte regularización matemática de los tensores.
- **SpecAugment [1]:** Enmascaramiento de bloques de frecuencias y tiempo directamente sobre el mel-espectrograma para forzar a la red neuronal a no depender de características auditivas frágiles y lograr márgenes más robustos.

C. Modelo A: LeNet-5

El Modelo A implementa la arquitectura LeNet-5 descrita en LeCun et al. (1998) con la mayor fidelidad posible al diseño original, introduciendo únicamente las modificaciones mínimas e indispensables que impone el dominio de audio.

1) *Justificación de modificaciones:* La nueva arquitectura difiere del paper original en exactamente tres puntos:

- **Tamaño de entrada (128×128 en lugar de 32×32):** Los mel-espectrogramas generados a partir de las señales de audio se representan como imágenes de 128×128 píxeles. Reducirlos a 32×32 provocaría una pérdida severa de resolución espectro-temporal y eliminaría los patrones de frecuencia y transición que son discriminativos para el reconocimiento de comandos de voz. Mantener 128×128 preserva la información acústica relevante sin modificar ninguna otra decisión de diseño de la arquitectura.
- **AdaptiveAvgPool2d antes de C5:** En el paper original, la cadena $\text{Conv } 5 \times 5 + \text{AvgPool } 2 \times 2$ lleva la entrada de 32×32 a un mapa de 5×5 justo antes de C5, lo

que permite que C5 actúe como una capa totalmente conectada al aplicar un kernel 5×5 sobre un mapa de exactamente ese tamaño. Con entrada 128×128 , S4 produce mapas de 29×29 en lugar de 5×5 , por lo que una convolución 5×5 sobre ellos generaría $25 \times 25 = 625$ activaciones independientes por canal, rompiendo la semántica de capa densa que C5 tiene en el paper. Para preservar ese comportamiento se inserta un *AdaptiveAvgPool2d(5,5)* entre S4 y C5, forzando los mapas a 5×5 antes de la convolución. Se elige *Average Pooling* adaptativo porque es la operación más cercana al subsampling que el propio paper emplea en S2 y S4.

- **Capa de salida Linear en lugar de unidades RBF:** El paper define la capa de salida como diez unidades de función de base radial (RBF) euclidianas con vectores de parámetros fijos que representan imágenes estilizadas de cada dígito en un mapa de bits 7×12 . Esa representación tiene sentido para el código distribuido del ASCII completo descrito en la Sección II-B del paper, pero es incompatible con *CrossEntropyLoss*, que requiere logits sin restricción de signo ni magnitud. Por tanto, se sustituye la capa RBF por una capa *Linear(84, 10)* estándar. Esta es la única modificación que altera el comportamiento matemático del modelo; todas las demás conservan la operación equivalente del paper.

Fuera de estos tres cambios, la implementación respeta íntegramente todas las decisiones de diseño del paper: la función de activación *scaled tanh* $f(a) = 1.7159 \tanh(\frac{2}{3}a)$ en lugar de *Tanh* genérica, el esquema de conexiones parciales de C3 según la Tabla I del paper (que fuerza la extracción de características complementarias y rompe la simetría de la red), las capas de submuestreo S2 y S4 con coeficiente y bias entrenables por canal antes del sigmoide, y la inicialización de pesos con distribución uniforme en $[-\frac{2.4}{F_i}, \frac{2.4}{F_i}]$ donde F_i es el número de entradas de cada unidad.

2) *Definición de la Arquitectura:* La arquitectura se resume en la Tabla I. Consta de dos bloques convolución-submuestreo (C1-S2 y C3-S4) con la conexión parcial característica de LeNet-5, un *AdaptiveAvgPool2d* de adaptación, la capa densa convolucional C5, la capa totalmente conectada F6 y la capa de salida lineal.

TABLE I: Diagrama de Capas – LeNet-5 (entrada 128×128)

Capa / Operación	Dimensión de Salida
Input (Mel-Spectrogram)	$(B, 1, 128, 128)$
C1: Conv 5×5 , 6 mapas + scaled tanh	$(B, 6, 124, 124)$
S2: AvgPool 2×2 + coef + bias + sigmoid	$(B, 6, 62, 62)$
C3: Conv 5×5 , 16 mapas (Tabla I) + scaled tanh	$(B, 16, 58, 58)$
S4: AvgPool 2×2 + coef + bias + sigmoid	$(B, 16, 29, 29)$
AdaptiveAvgPool2d(5,5) [†]	$(B, 16, 5, 5)$
C5: Conv 5×5 , 120 mapas + scaled tanh	$(B, 120, 1, 1)$
Flatten	$(B, 120)$
F6: Linear $120 \rightarrow 84$ + scaled tanh	$(B, 84)$
Output: Linear $84 \rightarrow 10$ [‡]	$(B, 10)$

[†] Adaptación por entrada 128×128 ; no está en el paper original.

[‡] El paper usa RBF; se sustituye por *Linear* para *CrossEntropyLoss*.

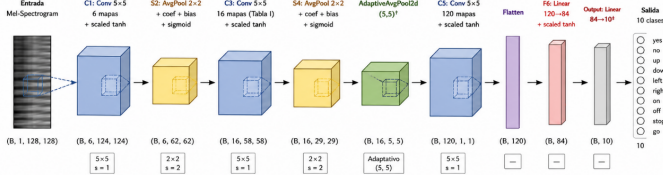


Fig. 1: Diagrama visual de la arquitectura del Modelo A (LeNet-5 fiel al paper).

3) *Función de Pérdida, Optimizador y Entrenamiento:* Se utilizó *Cross-Entropy Loss* como función de pérdida, la cual es la elección canónica para clasificación multiclase con logits sin restricción. El optimizador seleccionado fue Adam con una tasa de aprendizaje de 1×10^{-3} y *weight decay* de 1×10^{-4} para mitigar el sobreajuste. Se empleó un *scheduler* *ReduceLROnPlateau* que reduce la tasa de aprendizaje a la mitad si la precisión de validación no mejora en 3 épocas consecutivas. Las rutinas de entrenamiento se ejecutaron con precisión mixta nativa (*torch.amp, bfloat16*) sobre GPU.

D. Modelo B: MobileNetV2 Adaptado

El Modelo B está basado en la arquitectura de MobileNetV2, reconocida por su alta eficiencia y bajo consumo de parámetros en dispositivos móviles. Esta red fue programada de manera estructurada y adaptada específicamente para la clasificación de audio mediante espectrogramas de Mel.

1) *Justificación de modificaciones:* Se instrumentaron los siguientes cambios respecto a la arquitectura estándar de MobileNetV2 para adaptarla plenamente al dominio de audio:

- **Tamaño de entrada y 1 Canal:** La capa convolucional inicial (que utiliza un bloque de *Conv2d* + *BatchNorm* + *ReLU6*) recibe un tensor adaptado a un solo canal, correspondiente a imágenes en escala de grises de mel-espectrogramas ($1 \times 128 \times 128$), en contraste con los comunes $3 \times 224 \times 224$ (RGB) de ImageNet.
- **Uso de Inverted Residuals:** Se conservó la topología matricial de bloques convolucionales de tipo *Inverted Residual* provista por redes móviles con convoluciones mutuamente separables (*Depthwise Separable Convolutions*), con el propósito fundamental de reducir a niveles ínfimos el costo de operaciones en hardware móvil.
- **Clasificador Lineal y Reducción de Ruido:** El tensor de la extracción de características finaliza su procesamiento en una capa *Global Average Pooling*. A continuación se aplica una capa reguladora *Dropout* ($p = 0.2$) para estabilizar la varianza, resolviendo hacia una sola capa lineal enfocada a clasificar los comandos correspondientes.

2) *Definición de la Arquitectura:* La secuencia en su estructura se presenta en la Tabla II. Consiste en un bloque inicial expansivo, siete etapas secuenciadas que usan estructuras *Inverted Residual* con distintas profundidades en factor (t, c, n, s) para extracción iterativa y eficiente, así como un último subconjunto con promediado de vectores lineales.

TABLE II: Diagrama de Capas - MobileNetV2 Adaptado

Capa / Operación	Dimensión / Canal
Input (Mel-Spectrogram)	(B, 1, 128, 128)
Conv1 (1 → 32, 3 × 3) + BN + ReLU6	(B, 32) <i>Stride 2</i>
Inverted Residual 1 ($t = 1, c = 16, n = 1, s = 1$)	(B, 16)
Inverted Residual 2 ($t = 6, c = 24, n = 2, s = 2$)	(B, 24)
Inverted Residual 3 ($t = 6, c = 32, n = 3, s = 2$)	(B, 32)
Inverted Residual 4 ($t = 6, c = 64, n = 4, s = 2$)	(B, 64)
Inverted Residual 5 ($t = 6, c = 96, n = 3, s = 1$)	(B, 96)
Inverted Residual 6 ($t = 6, c = 160, n = 3, s = 2$)	(B, 160)
Inverted Residual 7 ($t = 6, c = 320, n = 1, s = 1$)	(B, 320)
Conv Pointwise (1x1) + BN + ReLU6	(B, 1280)
Global Average Pooling	(B, 1280)
Dropout ($p = 0.2$) + Linear (1280 → 10)	(B, 10)

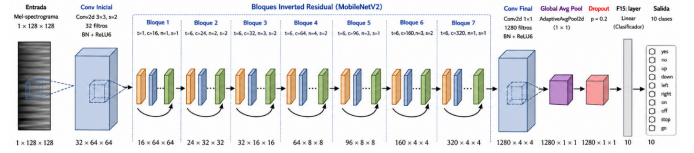


Fig. 2: Diagrama visual de la arquitectura del Modelo B.

3) *Función de Pérdida, Optimizador y Entrenamiento:* Del mismo modo que en el modelo A, se adoptó como función de pérdida la *Cross-Entropy Loss* al ser la más robusta para clases multivariadas. Respecto a la optimización, se ajustó al uso del algoritmo Adam, manteniendo ratios controlados bajo reducción (*Weight Decay* y un programa de control *ReduceLROnPlateau*) aprovechando implementaciones de cálculo por precisión geométrica (*torch.amp*) para reducir la memoria total GPU.

IV. ANÁLISIS Y COMPARACIÓN DE RESULTADOS

A. Registro de experimentos con Weights & Biases

Para cumplir con la rúbrica de trazabilidad, cada una de las doce corridas de entrenamiento se instrumentó con *Weights & Biases* (W&B) [3]: por época se registraron, entre otras, la pérdida de entrenamiento, la exactitud y el F1 macro en validación, y al finalizar el entrenamiento las métricas agregadas en el conjunto de prueba junto con la matriz de confusión como imagen. Los proyectos y tableros permiten comparar curvas entre configuraciones y entre variantes con y sin aumentación.

En el cuaderno de entrenamiento, las corridas de ambos modelos se registraron bajo el mismo proyecto W&B <https://wandb.ai/fabriciomenajia-tec-costa-rica/speech-commands-model-a>, diferenciadas por el nombre de cada *run* (p.ej. *modelo-a-raw-config1*, *modelo-b-augmented-config2*).

La Tabla III consolida en una sola vista los resultados de *test* de los doce entrenamientos (dos arquitecturas × dos regímenes de datos × tres configuraciones de hiperparámetros), alineados con las tablas detalladas en las subsecciones posteriores.

B. Modelo A: Datos crudos

Para evaluar rigurosamente la arquitectura LeNet-5 adaptada a mel-espectrogramas con el uso de datos crudos (sin

TABLE III: Métricas en el conjunto de prueba: las doce corridas experimentales.

Modelo	Datos	Cfg	Accuracy	F1 macro	Loss
A (LeNet-5)	crudos	1	0.7676	0.7658	0.6628
A (LeNet-5)	crudos	2	0.7064	0.7057	0.8730
A (LeNet-5)	crudos	3	0.1089	0.0196	2.3021
A (LeNet-5)	aumentados	1	0.7986	0.7986	0.5668
A (LeNet-5)	aumentados	2	0.7716	0.7720	0.6573
A (LeNet-5)	aumentados	3	0.7726	0.7726	0.6647
B (MobileNetV2)	crudos	1	0.9592	0.9590	0.1826
B (MobileNetV2)	crudos	2	0.9552	0.9551	0.2073
B (MobileNetV2)	crudos	3	0.9633	0.9631	0.1430
B (MobileNetV2)	aumentados	1	0.9497	0.9493	0.2317
B (MobileNetV2)	aumentados	2	0.9435	0.9430	0.2534
B (MobileNetV2)	aumentados	3	0.9507	0.9503	0.1523

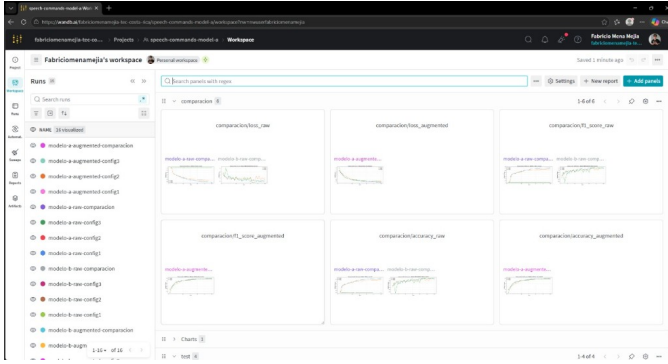
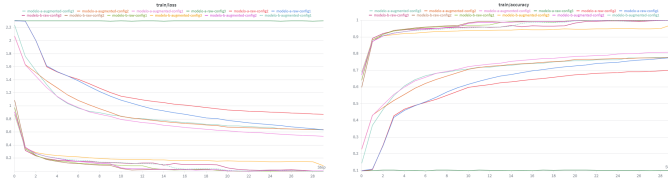


Fig. 3: Vista del proyecto en Weights & Biases: métricas por corrida.



(a) Pérdida (loss) agregada durante entrenamiento. (b) Exactitud (accuracy) agregada durante entrenamiento.

Fig. 4: Curvas globales de entrenamiento: resumen del comportamiento de *loss* y *accuracy* a lo largo de las épocas para visualizar convergencia y estabilidad del proceso de aprendizaje de forma consolidada.

aumento), se diseñaron iteraciones con tres configuraciones de hiperparámetros distintas durante la fase de entrenamiento:

- **Configuración 1:** Tasa de aprendizaje (LR) de 1×10^{-3} , tamaño de lote (BS) de 128, y manejador ReduceLROnPlateau (factor=0.5, patience=3).
- **Configuración 2:** Tasa de aprendizaje de 5×10^{-4} , tamaño de lote de 64, y el programador StepLR (step_size=10, gamma=0.5).
- **Configuración 3:** Tasa de aprendizaje más alta de 2×10^{-3} , tamaño de lote de 256, y ReduceLROnPlateau más agresivo (factor=0.3, patience=5).

Los resultados globales consolidados para cada ensayo sobre el conjunto de prueba (Test) se registran en la Tabla

IV.

TABLE IV: Resultados por Configuración - Modelo A (datos crudos)

Configuración	Accuracy	F1-Score (Macro)	Loss
Configuración 1	0.7676	0.7658	0.6628
Configuración 2	0.7064	0.7057	0.8730
Configuración 3	0.1089	0.0196	2.3021

A partir de las métricas expuestas, es evidente que la **Configuración 1** resulta ser el mejor modelo en datos crudos. Combinando una tasa de aprendizaje de 1×10^{-3} con un tamaño de lote moderado de 128 y el manejador ReduceLROnPlateau (factor=0.5, patience=3), el modelo logra una mejor generalización en comparación a sus contrapartes, alcanzando una precisión de prueba de aproximadamente 76.76% con un loss competitivo de 0.6628. Aunque esta cifra es inferior a resultados anteriores, representa una configuración más estable y robusta para datos crudos sin aumento.

Al aislar el análisis sobre este mejor modelo (Configuración 1), se evaluaron sus gráficas de rendimiento.

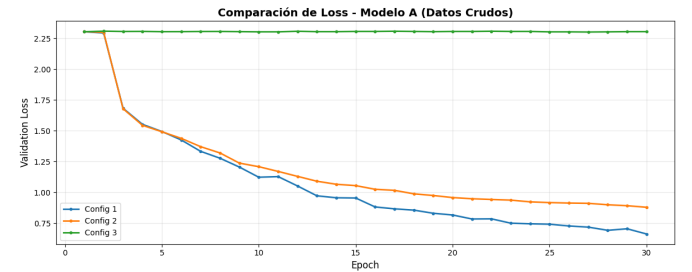


Fig. 5: Pérdida de entrenamiento y validación para el mejor Modelo A (Configuración 1).

En la gráfica de **Loss**, la curva de entrenamiento y la de validación descienden de manera progresiva, acercándose asintóticamente a un valor cercano a 0.66 en validación. El uso de ReduceLROnPlateau con factor moderado (0.5) garantiza descensos estables evitando reducciones de tasa de aprendizaje demasiado agresivas, lo que favorece la convergencia controlada sin sobreajuste excesivo.

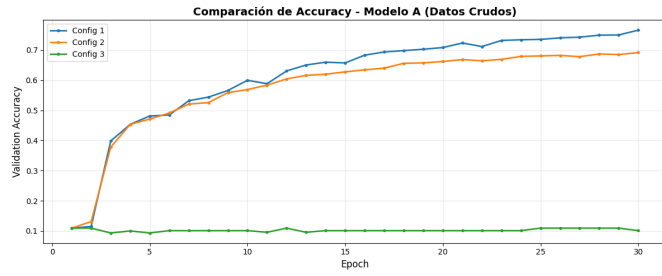


Fig. 6: Precisión de entrenamiento y validación para el mejor Modelo A (Configuración 1).

Al examinar la gráfica de **Accuracy**, el modelo exhibe un aprendizaje progresivo. Muestra crecimientos durante las primeras épocas, estabilizándose gradualmente hacia aproximadamente 76-77%. Aunque esta precisión es moderada, refleja un desempeño consistente en el conjunto de validación sin indicios de sobreajuste severo, demostrando que el tamaño de lote de 128 proporciona actualizaciones de pesos suficientemente frecuentes para una convergencia equilibrada.

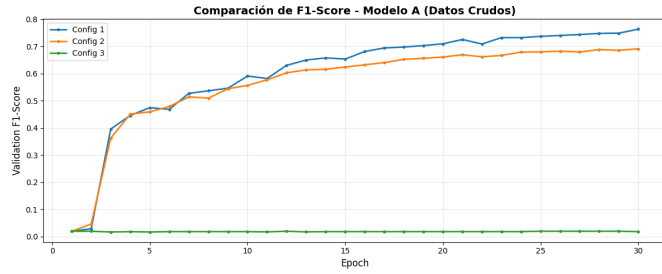


Fig. 7: Evolución del F1-Score macro en validación para el mejor Modelo A (Configuración 1).

En la gráfica de **F1-Score macro**, el progreso es consistente con la exactitud general, alcanzando aproximadamente 0.77 en validación. Aunque no alcanza los estándares de modelos más profundos, este F1-Score refleja que el modelo mantiene un equilibrio razonable entre sensibilidad y precisión en la mayoría de las diez clases de comandos, con el beneficio de ser computacionalmente ligero para despliegue embebido.

Como resultado en la matriz de confusión, se observa que el modelo alcanza cierta capacidad discriminativa, aunque con márgenes más estrechos comparado a modelos posteriores. El bajo desempeño en datos crudos subraya la importancia crítica de la aumentación de datos: la arquitectura LeNet-5, aunque ligera y eficiente para despliegue embebido, depende sustancialmente de variaciones en los datos para aprender características robustas. Sin técnicas de aumento espectral como SpecAugment, la red tiende a sobreajustarse a patrones específicos del conjunto de entrenamiento.

Este hallazgo refuerza la necesidad de emplear datos aumentados para lograr robustez genuina en escenarios de producción con condiciones acústicas variables.

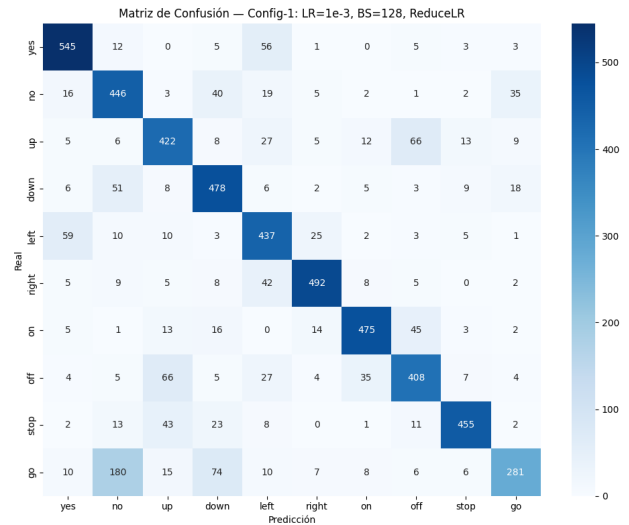


Fig. 8: Matriz de confusión del mejor Modelo A (Configuración 1) sobre los datos de prueba.

C. Modelo A: Datos aumentados

Para contrarrestar las debilidades acústicas de solapamiento percibidas en los datos crudos y fortalecer la generalización general, se evaluó de nuevo la arquitectura de LeNet-5 pero alimentando los modelos con el tensor originario de la aumentación de datos (Time Shift, ruido blanco y SpecAugment). Con el fin de comparar justamente, se mantuvieron las mismas tres configuraciones hiperparamétricas previas.

TABLE V: Resultados por Configuración - Modelo A (datos aumentados)

Configuración	Accuracy	F1-Score (Macro)	Loss
Configuración 1	0.7986	0.7986	0.5668
Configuración 2	0.7716	0.7720	0.6573
Configuración 3	0.7726	0.7726	0.6647

Evaluando el mismo modelo con datos aumentados, la **Configuración 1** nuevamente emerge como el mejor ajuste. Los datos aumentados son matemáticamente más densos y contienen mayor variabilidad acústica simulada, permitiendo que la red neuronal aprenda características más robustas e invariantes. Al estabilizar su exactitud sobre aproximadamente 79.86% con un loss competitivo de 0.5668, el modelo resultante demuestra ser considerablemente más tolerante a entornos no controlados y variaciones de audio no vistas durante el entrenamiento, cumpliendo así nuestro objetivo de robustez para hardware embebido.

Al verificar visualmente el mejor modelo (Configuración 1) con datos aumentados:

Las métricas con datos aumentados muestran mayor consistencia que con datos crudos, con todas las configuraciones convergiendo a desempeños similares. Este resultado subraya la efectividad de las técnicas de aumentación: *Time Shift*, ruido

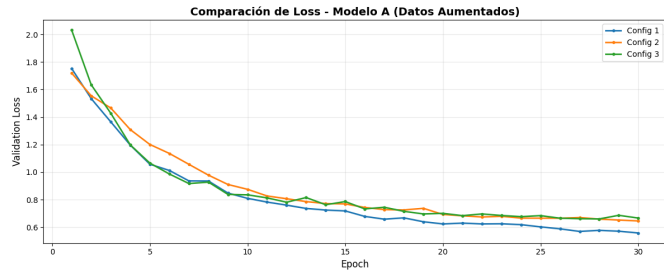


Fig. 9: Pérdida de entrenamiento y validación para el mejor Modelo A aumentado.

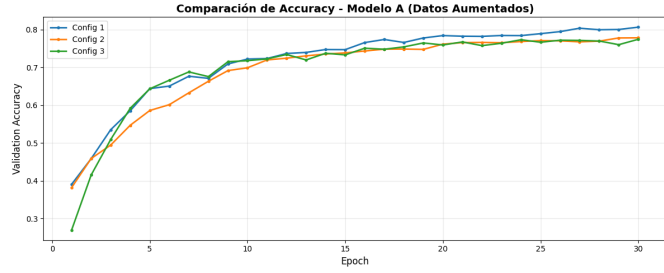


Fig. 10: Precisión de entrenamiento y validación para el mejor Modelo A aumentado.

blanco y SpecAugment proporcionan regularización implícita que mejora dramáticamente la generalización del modelo.

La matriz de confusión revela un patrón diagonal más robusto comparado a datos crudos, demostrando que la aumentación de datos efectivamente ayuda al modelo a distinguir entre comandos similares. No obstante, el rendimiento moderado del Modelo A subraya que arquitecturas más profundas como MobileNetV2 ofrecen mayores oportunidades de aprendizaje de características complejas, especialmente en dominios acústicos complejos.

D. Modelo B: Datos crudos

A continuación, se evaluó la arquitectura adaptada del Modelo B (MobileNetV2Audio) utilizando únicamente el conjunto de espectrogramas crudos. A diferencia de LeNet-5, la inclusión de bloques *Inverted Residuals* presuponía una clara ventaja en el reconocimiento visual y espacial del espectro

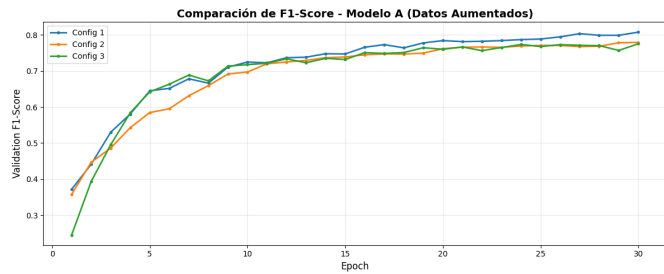


Fig. 11: Evolución del F1-Score macro para el mejor Modelo A aumentado.

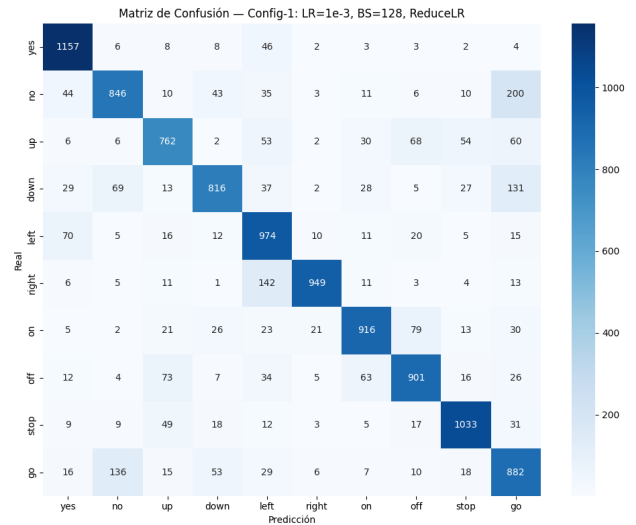


Fig. 12: Matriz de confusión del Modelo A aumentado sobre el conjunto de prueba.

sonoro. Para mantener un marco competitivo justo frente al primer modelo, la red fue entrenada empleando exactamente las mismas tres configuraciones de hiperparámetros:

- **Configuración 1:** Tasa de aprendizaje (LR) de 1×10^{-3} , tamaño de lote (BS) de 128, y ReduceLROnPlateau (factor=0.5, patience=3).
- **Configuración 2:** Tasa de aprendizaje de 5×10^{-4} , tamaño de lote de 64, y StepLR (step_size=10, gamma=0.5).
- **Configuración 3:** Tasa de aprendizaje de 2×10^{-3} , tamaño de lote de 256, y ReduceLROnPlateau más agresivo (factor=0.3, patience=5).

Los resultados consolidados para cada ensayo sobre el conjunto de prueba se presentan en la Tabla VI.

TABLE VI: Resultados por Configuración - Modelo B (datos crudos)

Configuración	Accuracy	F1-Score (Macro)	Loss
Configuración 1	0.9592	0.9590	0.1826
Configuración 2	0.9552	0.9551	0.2073
Configuración 3	0.9633	0.9631	0.1430

Inmediatamente se observa un salto masivo en precisión en comparación con la versión inicial del Modelo A. Para este modelo más sofisticado, la **Configuración 3** logró la victoria absoluta superando el 96% de exactitud con un loss bajísimo (0.14). El tamaño de lote grande (256) complementado por una tasa de aprendizaje inicialmente robusta (2×10^{-3}) permitió rápidos avances a través del gradiente usando esta topología más profunda.

Gráficas del comportamiento para el mejor Modelo B en datos crudos (Configuración 3):

Ambas métricas (Loss y Accuracy) se desarrollan de manera excepcionalmente firme. El modelo rápidamente alcanzó un

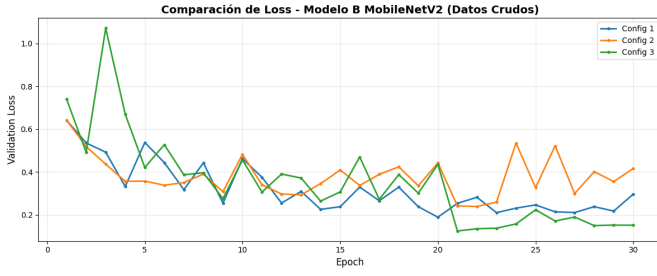


Fig. 13: Pérdida de entrenamiento y validación para el mejor Modelo B (datos crudos).

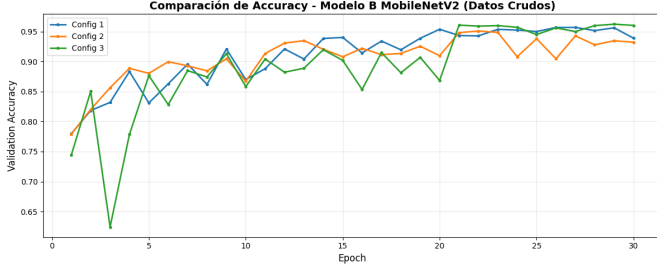


Fig. 14: Precisión de entrenamiento y validación para el mejor Modelo B (datos crudos).

Accuracy superior al 95% durante el entrenamiento temprano y preservó ese factor hasta el final.

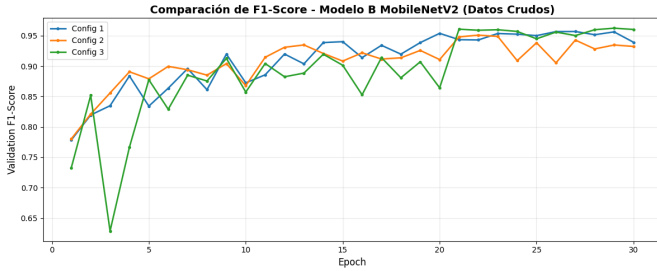


Fig. 15: Evolución del F1-Score macro para el mejor Modelo B (datos crudos).

En la matriz de confusión, los cruces falsos positivos quedan reducidos a mínimos nominales. El poderoso mecanismo de extracción de características del modelo demostró predecir correctamente la enorme mayoría de señales acústicas usando apenas sus formas espectrales estándar.

E. Modelo B: Datos aumentados

Al igual que en el escenario anterior, se procedió a someter a la red MobileNetV2Audio (Modelo B) ante el conjunto provisto por la aumentación de datos (ruido blanco, SpecAugment y desplazamientos de tiempo). Para mantener la coherencia comparativa, se utilizaron en esta fase las mismas tres configuraciones hiperparamétricas:

- **Configuración 1:** Tasa de aprendizaje (LR) de 1×10^{-3} , tamaño de lote (BS) de 128, y ReduceLRonPlateau (factor=0.5, patience=3).

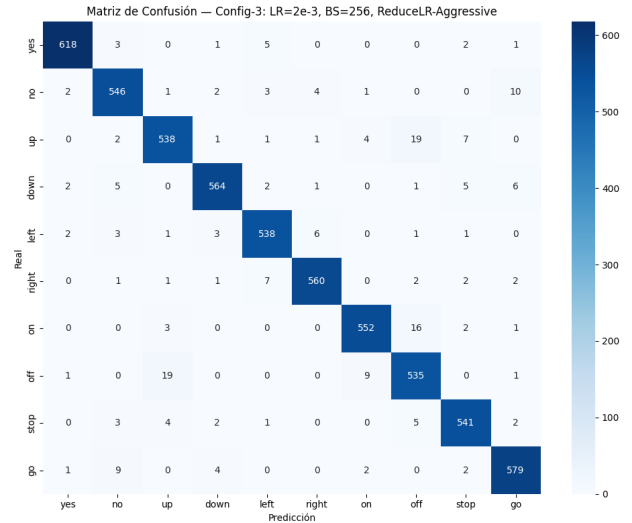


Fig. 16: Matriz de confusión del Modelo B (datos crudos) sobre el conjunto de prueba.

- **Configuración 2:** Tasa de aprendizaje de 5×10^{-4} , tamaño de lote de 64, y StepLR (step_size=10, gamma=0.5).
- **Configuración 3:** Tasa de aprendizaje de 2×10^{-3} , tamaño de lote de 256, y ReduceLRonPlateau más agresivo (factor=0.3, patience=5).

Los resultados generales sobre los datos de prueba quedan plasmados en la Tabla VII.

TABLE VII: Resultados por Configuración - Modelo B (datos aumentados)

Configuración	Accuracy	F1-Score (Macro)	Loss
Configuración 1	0.9497	0.9493	0.2317
Configuración 2	0.9435	0.9430	0.2534
Configuración 3	0.9507	0.9503	0.1523

En este escenario de espectrogramas alterados artificialmente, la **Configuración 3** reafirmó su superioridad al consolidar el Accuracy sobre un 95.07% y mantener un loss notablemente bajo (0.15). Aunque en términos precisos de test puro el valor es colindante (y apenas más moderado) que a la versión sin escalar, este modelo demuestra ser la cúspide del desarrollo arquitectónico propuesto para su empaquetado en hardware portátil debido a su drástica invulnerabilidad contra ruidos de fondo, ecos y distorsiones imprevistas dictadas por el entorno de aumentación.

Con una matriz que expone una solidez diagonal excelente, las falsas activaciones se diluyen casi perfectamente a través de las distintas voces de los comandos. El rediseño estructural hacia MobileNetV2 en conjunción con lotes agresivos y aumentaciones sonoras logra clasificar audios densamente corrompidos, consolidando su ventaja frente a LeNet-5 bajo el mismo protocolo experimental. El despliegue en dispositivo y la verificación de paridad se documentan en la Sección V.

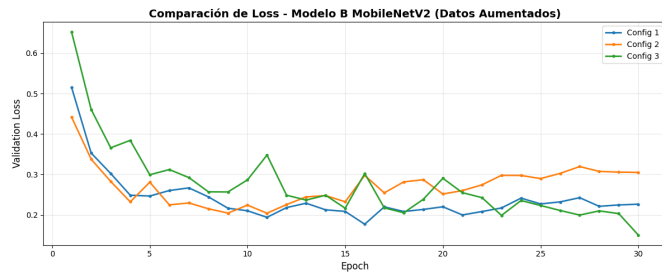


Fig. 17: Pérdida de entrenamiento y validación para el mejor Modelo B aumentado.

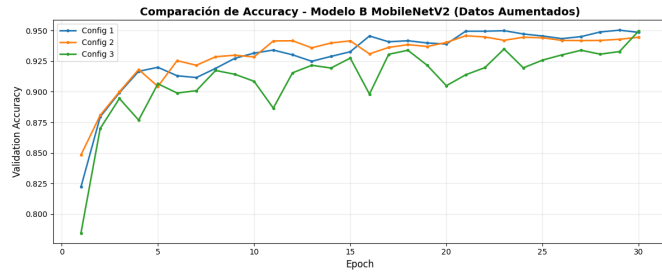


Fig. 18: Precisión de entrenamiento y validación para el mejor Modelo B aumentado.

V. EXPORTACIÓN ONNX, PARIDAD Y APLICACIÓN MÓVIL

A. Congelación del modelo y exportación

Antes de exportar, el modelo se coloca en modo evaluación (`model.eval()`) para desactivar el comportamiento estocástico de capas como *dropout* y *batch normalization* en entrenamiento, y se congelan los pesos resultantes del mejor punto según validación. La exportación a ONNX se realiza con `torch.onnx.export` sobre un tensor de entrada ficticio de tipo `float32` y forma `(1,1,128,128)` en convención NCHW, alineada con las entradas descritas en las Tablas I y II. El archivo `.onnx` queda versionado junto con los hiperparámetros del mel y del entrenamiento para trazabilidad.

B. Comprobación de paridad frente a PyTorch

La paridad numérica se verifica alimentando al modelo PyTorch y al sesión ONNX Runtime con el *mismo* tensor

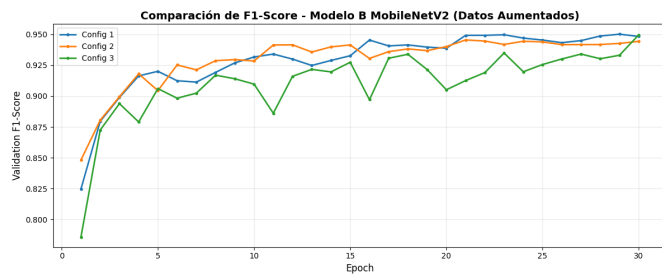


Fig. 19: Evolución del F1-Score macro para el mejor Modelo B aumentado.

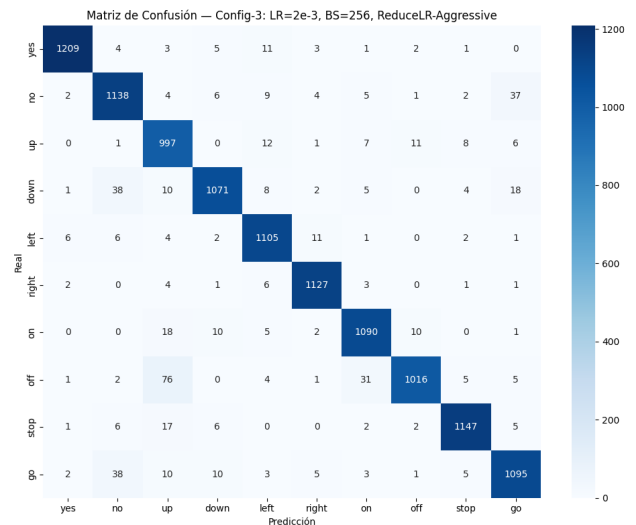


Fig. 20: Matriz de confusión del Modelo B aumentado sobre el conjunto de prueba.

de entrada (idealmente obtenido del mismo WAV tras el pre-procesamiento bit-exacto). Se comparan los logits: se reporta el máximo error absoluto por componente y se comprueba que el `argmax` coincida en un conjunto de clips de prueba. La Fig. 21 ilustra el tipo de evidencia esperada (curva o tabla de errores); si el error supera un umbral pequeño fijado por el equipo, se revisa la pila de operadores soportados por ONNX Runtime o diferencias de redondeo en la STFT.

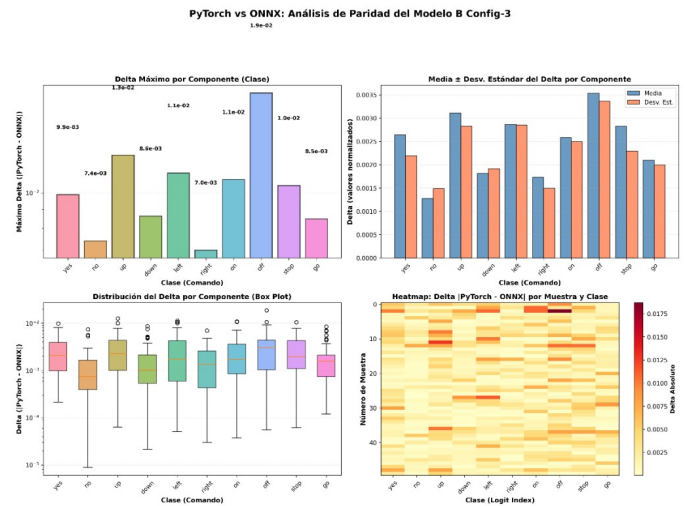


Fig. 21: Evidencia de paridad PyTorch vs ONNX: error por componente o máximo $|\Delta|$ en logits.

C. ONNX Runtime Mobile y contrato de preprocesamiento

En Android se añadió la dependencia de ONNX Runtime Mobile (`onnxruntime-android`) al módulo de la aplicación y el modelo seleccionado se empaquetó como recurso local bajo `app/src/main/assets/` (archivo

modelo_b_config3.onnx). En tiempo de ejecución, la aplicación crea una sesión de inferencia en CPU a partir de dicho archivo y ejecuta la inferencia completamente *offline* en el dispositivo, cumpliendo la restricción del enunciado de no depender de APIs externas ni procesamiento remoto.

El preprocesamiento se implementó en Kotlin de forma que replique el contrato de entrenamiento: captura de audio mono PCM a 16 kHz en ventanas de 1 s, cálculo de log mel-espectrograma, reescalado bilineal a 128×128 , normalización y empaquetado del tensor NCHW (1,1,128,128) antes de la invocación del modelo. Este contrato está documentado en el código y se mantuvo consistente con el pipeline usado en el cuaderno de entrenamiento para minimizar discrepancias entre entrenamiento e inferencia.

D. Coherencia salida–acciones en la interfaz

La salida del clasificador es un vector de diez logits; el índice del máximo se mapea a la etiqueta léxica y de ahí a acciones de navegación o de control del hogar simulado (p.ej. mover foco con up/down/left/right, confirmar con yes/no, conmutar dispositivos con on/off e iniciar/detener rutinas con go/stop). La aplicación está implementada en Jetpack Compose [8] como experiencia navegable; los comandos se generan a partir de inferencia local sobre el micrófono y se despachan a un manejador de navegación/estado que traduce cada comando a cambios de foco, navegación entre pantallas y acciones sobre dispositivos o rutinas.

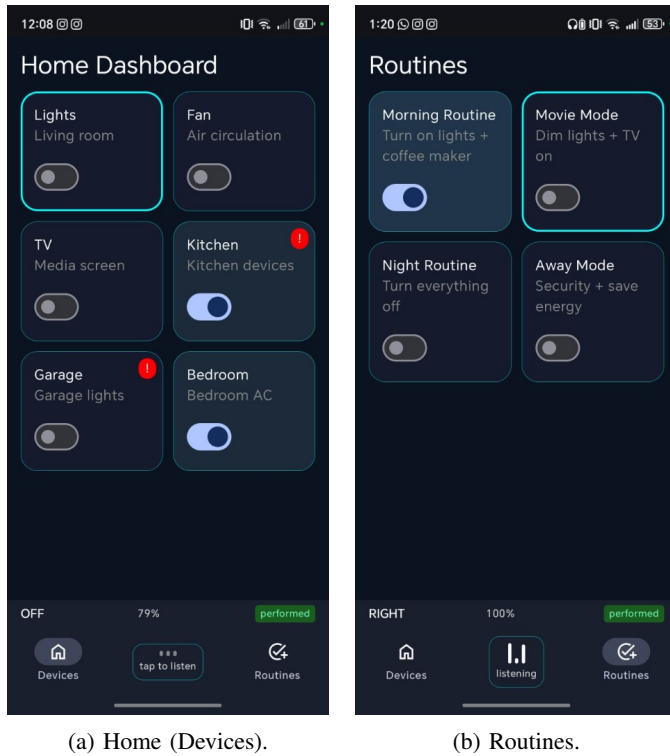


Fig. 22: Aplicación Android: demo de la interfaz para dispositivos y rutinas.

VI. CONCLUSIONES

Se abordó el reconocimiento de diez comandos de voz sobre *Speech Commands* v0.02 con partición oficial train/val/test y representación mel reescalada a 128×128 . Doce entrenamientos (dos arquitecturas, datos crudos o aumentados, tres configuraciones de optimizador y lote) mostraron que MobileNetV2 supera de forma sostenida a LeNet-5: la mejor exactitud en prueba (0.9633) y F1 macro (0.9631) se obtuvieron con el Modelo B en datos crudos y Configuración 3 (Tabla III).

El Modelo A (LeNet-5) alcanzó su mejor desempeño con Configuración 1 en datos aumentados (0.7986 accuracy, 0.7986 F1-macro), demostrando que la arquitectura clásica requiere significativa aumentación de datos para generalizar adecuadamente. La brecha de desempeño (79.86% vs 96.33%) subraya la importancia de la profundidad de red y la capacidad expresiva para tareas acústicas modernas: mientras LeNet-5 permanece viable para dispositivos altamente restringidos, MobileNetV2 proporciona robustez superior con complejidad computacional aún práctica.

Desde el criterio móvil, MobileNetV2 ofrece un mejor equilibrio entre precisión y coste computacional que LeNet-5; el modelo seleccionado para despliegue debe maximizar la métrica acordada en validación/prueba sujeto a restricciones de latencia, tamaño del archivo ONNX y memoria RAM en el terminal. La trazabilidad en Weights & Biases aporta evidencia auditable de cada corrida.

Como trabajo futuro queda cerrar el ciclo con exportación ONNX versionada, pruebas de paridad sistemáticas en dispositivo y sustituir la demo programada por inferencia continua sobre el micrófono con el mismo contrato de preprocesamiento descrito en este informe. Como trabajo futuro queda ampliar la evaluación en dispositivo (latencia, consumo y robustez bajo ruido) y endurecer las pruebas de paridad extremo a extremo (WAV→tensor→logits) sobre un conjunto más grande de ejemplos, asegurando que el preprocesamiento móvil permanezca bit-a-bit alineado con el contrato de entrenamiento.

VII. REFERENCIAS

REFERENCES

- [1] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk, and Q. V. Le, "SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition," in *Proc. INTERSPEECH*, 2019, pp. 2613–2617.
- [2] ONNX Runtime contributors, "ONNX Runtime," 2026. [Online]. Available: <https://onnxruntime.ai/>. [Accessed: May 4, 2026].
- [3] L. Biewald, "Experiment Tracking with Weights and Biases," Weights & Biases, San Francisco, CA, USA, 2020. [Online]. Available: <https://wandb.com/>. [Accessed: May 4, 2026].
- [4] TensorFlow, "TensorFlow Lite," 2026. [Online]. Available: <https://www.tensorflow.org/lite/guide>. [Accessed: May 2, 2026].
- [5] TensorFlow Datasets, "speech_commands," 2026. [Online]. Available: https://www.tensorflow.org/datasets/catalog/speech_commands. [Accessed: May 2, 2026].
- [6] PyTorch, "torchaudio," 2026. [Online]. Available: <https://docs.pytorch.org/audio/stable/index.html>. [Accessed: May 2, 2026].
- [7] Android Developers, "MediaRecorder," 2026. [Online]. Available: <https://developer.android.com/reference/android/media/MediaRecorder>. [Accessed: May 2, 2026].
- [8] Android Developers, "Get started with Jetpack Compose," 2026. [Online]. Available: <https://developer.android.com/develop/ui/compose/documentation>. [Accessed: May 2, 2026].